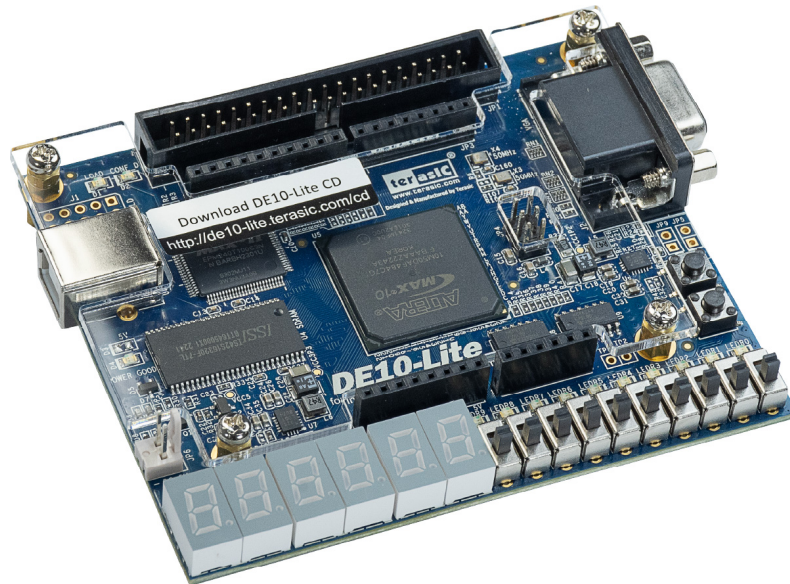


Rapport de Mini-Projet FPGA

Conception d'un Radar 2D

Carte DE10-Lite (Intel MAX10) & Nios II



Kevin CEPHISE Sarra SOLTAN :

Enseignant : Mr Yann DOUZE

Date : 16 janvier 2026



POLYTECH[®]
SORBONNE



SORBONNE
UNIVERSITÉ

1 Introduction et Objectifs

L'objectif de ce projet est de concevoir une architecture SoC-FPGA capable de cartographier une scène à l'aide d'un capteur ultrasonique et d'un servomoteur. Le système final est piloté par un processeur Nios II, qui orchestre l'acquisition des distances et le contrôle de la position angulaire.

Le développement se décompose en trois phases techniques :

1. **Conception IP** : Développement des modules VHDL pour le télémètre (avec interface Avalon) et le servomoteur.
2. **Intégration** : Assemblage du système sous Platform Designer et implémentation sur la carte DE10-Lite.
3. **Logiciel** : Programmation de la boucle de contrôle et d'affichage en C.

Le présent rapport se concentre sur la validation complète de la première brique du système : l'IP du télémètre ultrason.

2 Étape 1 : Implémentation de l'IP Télémètre Ultrason (HC-SR04)

2.1 Présentation du capteur et principe de fonctionnement

Le capteur utilisé est le module **HC-SR04**, dont les caractéristiques principales sont :

- Plage de mesure : 2 cm à 400 cm
- Résolution : 0,3 cm
- Angle de mesure : 15°

Principe de fonctionnement

Le capteur mesure une distance en évaluant le temps de vol d'un signal ultrasonore. Le fonctionnement se déroule en trois étapes :

1. Envoi d'une impulsion **Trigger** d'au moins 10 μ s.
2. Émission par le capteur d'une salve d'ondes ultrasons à 40 kHz.
3. Retour d'un signal **Echo** dont la durée est proportionnelle à la distance.

Un délai de **60 ms minimum** doit être respecté entre deux mesures pour éviter les interférences d'écho.

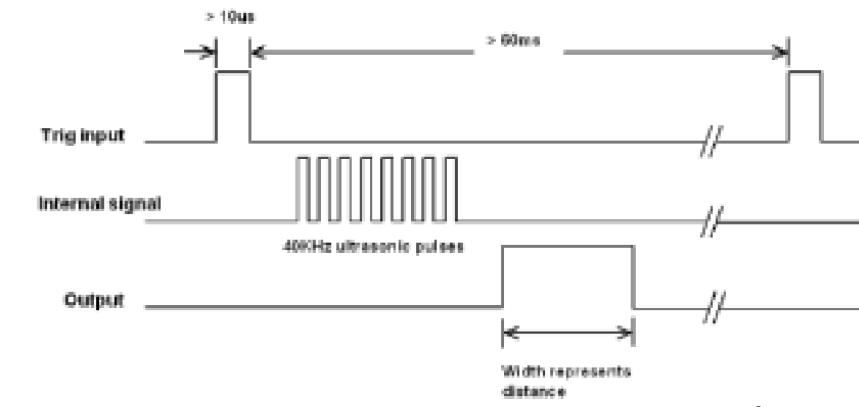


FIGURE 1 – Schéma du fonctionnement du signal Trigger/Echo

2.2 Conception et Simulation (Modelsim)

La première étape a consisté à décrire la **partie opérative** de l'IP en VHDL, gérant :

- La génération du signal `trig`.
- La mesure de la largeur du signal `echo`.
- La conversion de cette durée en distance (cm).

Un **testbench** a ensuite été réalisé sous Modelsim pour vérifier :

- la bonne génération du trigger,
- la prise en compte correcte du signal echo simulé,
- la mise à jour de la sortie `Dist_cm`.

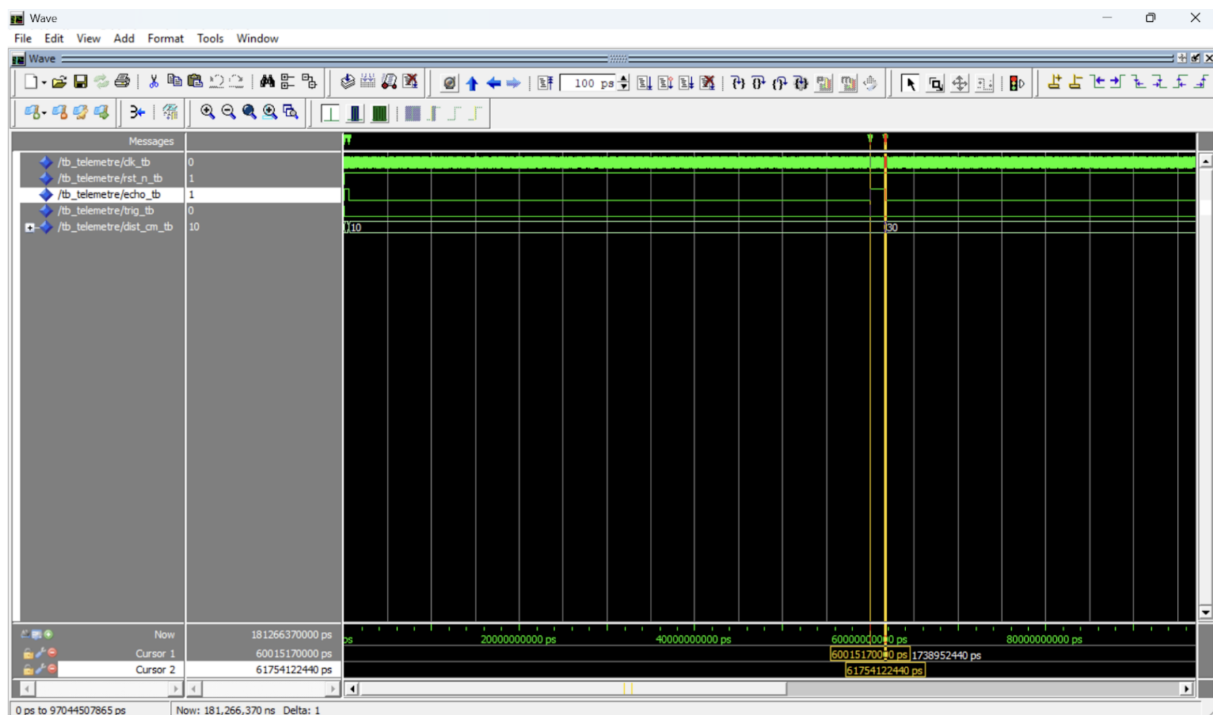


FIGURE 2 – Simulation Modelsim du télémètre : signaux `clk`, `trig`, `echo`, `Dist_cm`.

La simulation valide le bon respect des contraintes temporelles du capteur.

2.3 Test de l'IP sur carte DE10-Lite

Avant l'intégration dans un système Nios II, l'IP a été testée en mode **standalone** sur la carte DE10-Lite.

Configuration matérielle

L'IP a été déclarée comme *Top Level* dans Quartus. Les connexions FPGA → capteur ont été définies comme suit :

Signal IP	Périphérique DE10-Lite	Pin FPGA
Rst_n	Bouton KEY0	PIN_B8
CLK	Horloge 50 MHz	PIN_P11
Trig	GPIO[1]	PIN_W10
Echo	GPIO[3]	PIN_W9
Dist_cm	LEDs (LEDR 9..0)	PIN_A8...

TABLE 1 – Assignations des entrées/sorties du télémètre.

Le capteur a été alimenté en +5V via le connecteur GPIO JP1.

Résultats expérimentaux

Après programmation du FPGA, plusieurs distances connues ont été testées. La valeur binaire affichée sur les LEDs évolue correctement en fonction de l'obstacle.

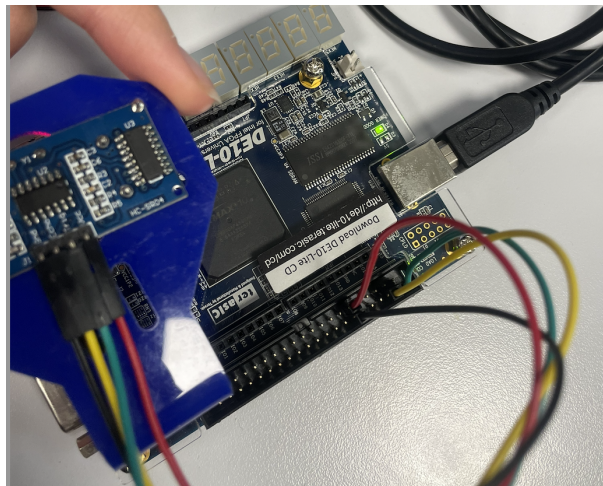


FIGURE 3 – Montage expérimental du télémètre sur DE10-Lite.

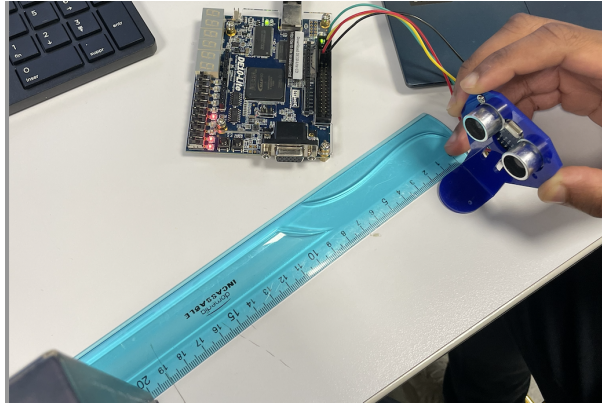


FIGURE 4 – Affichage de la distance mesurée sur les LEDs.

Ces tests confirment la fiabilité de l'IP télémètre. L'étape suivante consistera à ajouter l'interface Avalon pour l'intégration dans le système Nios II.

2.4 1.2.3 Intégration de l'IP Télémètre dans Platform Designer

Adaptation de l'interface VHDL (Avalon Memory Map) Afin d'intégrer le télémètre au système basé sur le processeur Nios II, nous avons modifié l'entité VHDL pour y ajouter une interface esclave Avalon-MM. Cette interface permet au processeur de lire directement la valeur de distance mesurée.

L'entité `Telemetre_us_Avalon` comporte désormais les signaux suivants :

- **Signaux Avalon (Esclave)** : `chipselct`, `Read_n`, `readdata(31..0)`.
- **Signaux Externes (Conduit)** : `trig`, `echo`, `Dist_cm`.
- **Signaux Système** : `clk`, `Rst_n`.

Validation par simulation de l'interface Avalon Une simulation Modelsim a été réalisée pour valider la lecture Avalon (Read Transfer). L'objectif était de vérifier que le bus `readdata` retourne correctement la valeur de distance lorsque `chipselct` et `Read_n` sont activés.

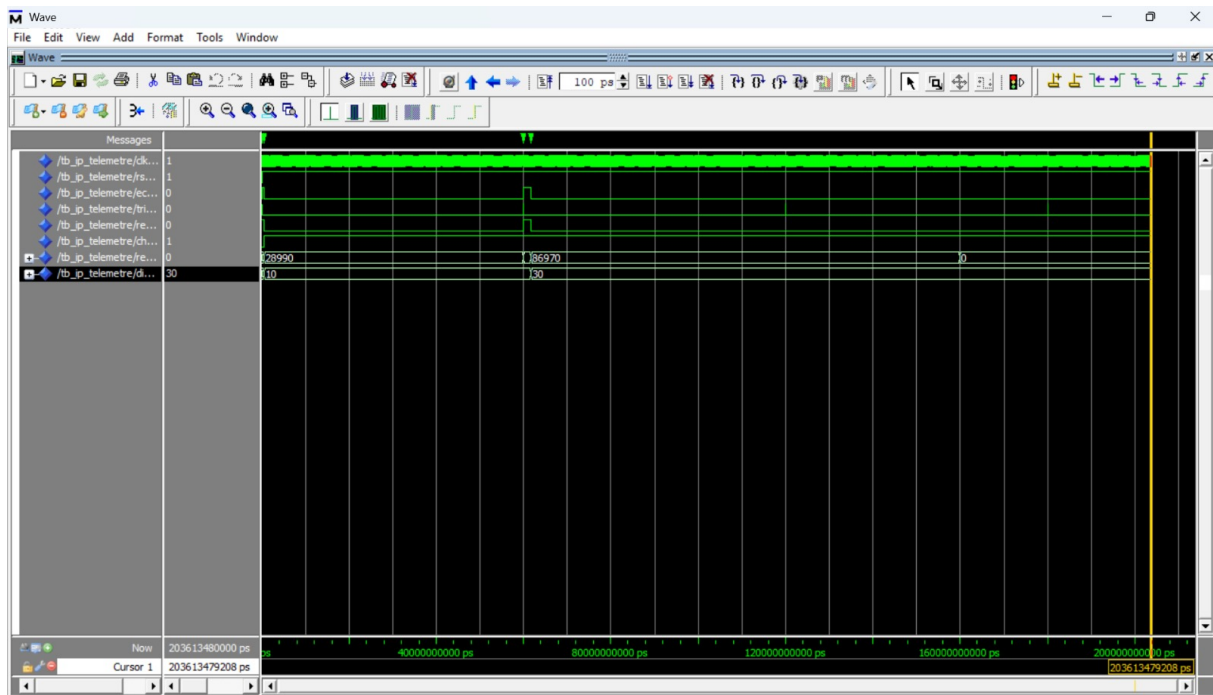


FIGURE 5 – Simulation du cycle de lecture Avalon pour l’IP Télémètre.

Commentaire : La simulation montre que l’on récupère correctement la valeur de distance sur le bus readdata en nombre de tour de clock.

Intégration dans Platform Designer (Qsys) Pour l’intégration, nous avons utilisé le projet source DE10_Lite_Computer_System, qui contient déjà un processeur Nios II configuré.

Les étapes suivies sont :

1. **Création du composant** : Ajout d’un composant via “New Component” et import du fichier VHDL.
2. **Mapping des signaux** : Connexion des signaux Avalon (horloge, reset, avalon_slave).
3. **Interface Conduit** : Création d’une interface conduit_end pour les signaux physiques trig, echo, Dist_cm.

Après la génération du système, le fichier top-level Quartus a été mis à jour afin de connecter les signaux du conduit aux broches du FPGA (GPIO, LEDs). Une compilation complète a ensuite été effectuée.

2.5 1.2.4 Programmation logicielle et test de l’IP

Développement du logiciel embarqué (C) Une fois le matériel configuré, nous avons développé un programme en C pour le processeur Nios II. Celui-ci lit périodiquement la valeur de distance mesurée et l’affiche dans le terminal ou sur les afficheurs 7 segments.

La lecture s’effectue via la macro Avalon :

```
IORD (IP_TELEMETRE_AVALON_BASE, 0)
```

Le programme réalise les opérations suivantes :

1. Lecture de la distance via l'IP.
2. Affichage de la valeur dans la console Nios II (`printf`).
3. Affichage sur les afficheurs 7 segments pour une lecture directe sur la carte.

Résultats et validation Le système complet a été testé en temps réel. La distance détectée est correctement transmise au processeur puis affichée.

La console Nios II affiche les mesures en centimètres, confirmant la bonne communication entre le processeur et l'IP Télémètre.

3 Étape 2 : Conception de l'IP Servomoteur

3.1 Fonctionnement du Servomoteur

Le contrôle d'un servomoteur repose sur l'envoi d'une impulsion périodique dont la largeur détermine l'angle mécanique. L'impulsion doit être répétée toutes les 10 à 20 ms.

Les correspondances angulaires standard sont généralement les suivantes :

- **1.00 ms** : position 0° (extrémité),
- **1.50 ms** : position 45° (milieu),
- **2.00 ms** : position 90° (autre extrémité).

Ainsi, la largeur de l'impulsion constitue le paramètre principal permettant de définir la position du moteur.

3.2 Conception de la Partie Opérationnelle du Composant Servomoteur

La partie opérationnelle de l'IP a été réalisée en VHDL afin de générer un signal PWM conforme aux positions demandées. Dans cette première version, l'IP est contrôlée directement à l'aide des interrupteurs de la DE10-Lite pour une validation standard.

Spécifications de l'entité

L'entité VHDL du servomoteur contient :

- **Entrées :**
 - `clk` : horloge système à 50 MHz,
 - `reset_n` : réinitialisation active à l'état bas,
 - `position` : valeur sur 8 ou 10 bits représentant l'angle en degrés.
- **Sortie :**
 - `commande` : signal PWM destiné à piloter le servomoteur.

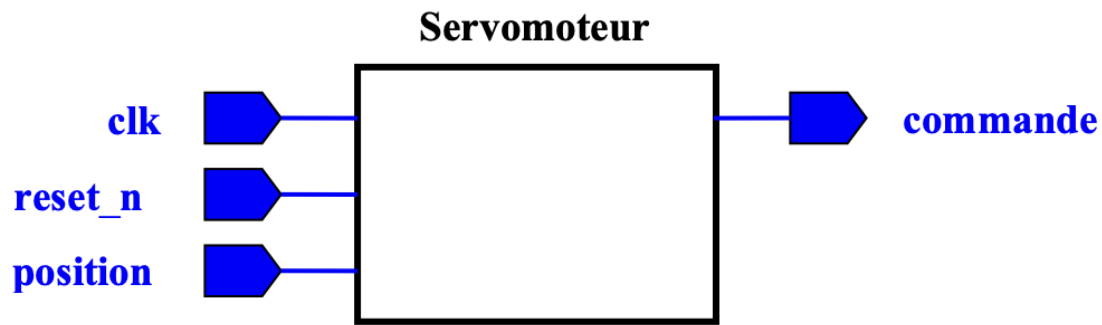


FIGURE 6 – Entité du servomoteur

3.3 Architecture de l'IP Servomoteur

L'architecture adoptée est de type comportementale et repose sur une Machine à États Finis (FSM). Elle comprend deux éléments principaux :

1. Génération du cycle périodique (20 ms) Un compteur mesure la période totale du PWM. Avec une horloge de 50 MHz :

$$20 \text{ ms} = 20 \times 10^{-3} \times 50 \times 10^6 = 1\,000\,000 \text{ cycles}$$

2. Conversion position → temps d'impulsion La largeur de l'impulsion est obtenue en appliquant une règle de trois dans la plage 1 ms–2 ms :

$$\text{Impulsion (cycles)} = \text{Impulsion}_{\min} + \left(\frac{\text{position}}{90} \right) \times (\text{Impulsion}_{\max} - \text{Impulsion}_{\min})$$

Avec :

$$\text{Impulsion}_{\min} = 0,5 \text{ ms} \times 50 \text{ MHz} = 25\,000$$

$$\text{Impulsion}_{\max} = 2,5 \text{ ms} \times 50 \text{ MHz} = 125\,000$$

3. Génération du signal commande La FSM génère :

- un état **HIGH** où `commande` = '1' pendant la durée calculée,
- un état **LOW** pour compléter la période de 20 ms.

3.4 Simulation et Validation (Modelsim)

Un testbench VHDL a été développé pour simuler le fonctionnement de l'IP Servomoteur.

Position (°)	Impulsion Attendue	Période
0	0,5 ms	20 ms
90	1.50 ms	20 ms
180	2.50 ms	20 ms

TABLE 2 – Validation des temps d’impulsion en simulation.

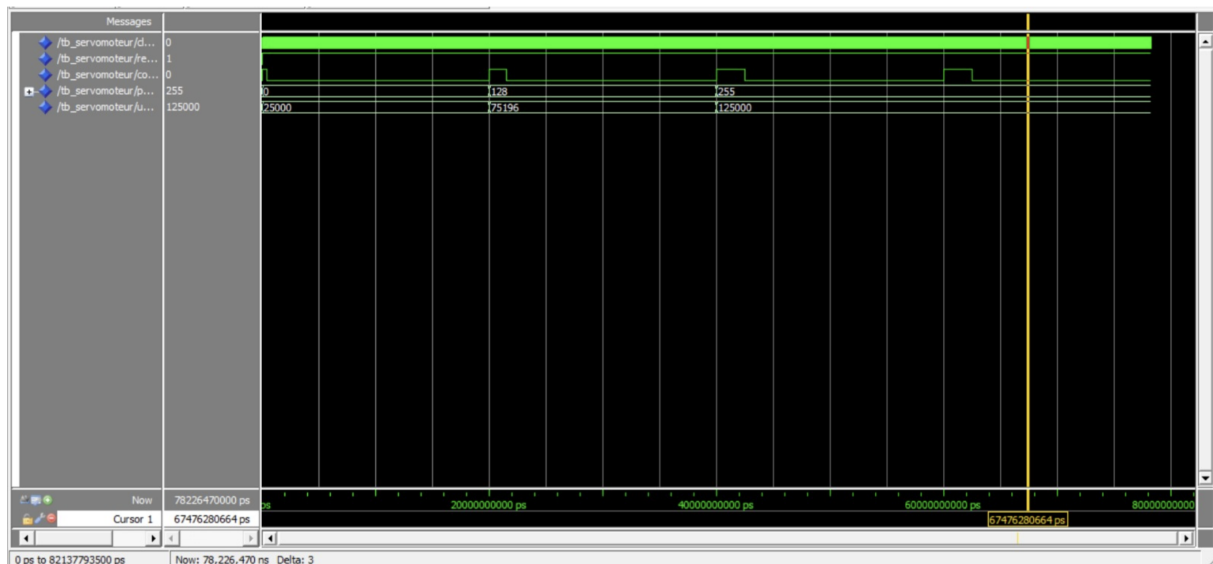


FIGURE 7 – Simulation du signal commande pour différentes positions.

La simulation a confirmé la précision des largeurs d’impulsion et leur répétition régulière.

3.5 Vérification sur Carte DE10-Lite

L’IP Servomoteur a ensuite été testée en mode standalone sur la carte DE10-Lite.

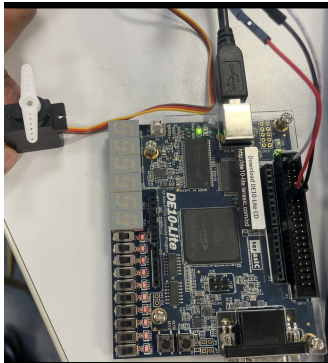
Connexions matérielles :

- clk : horloge 50 MHz,
- reset_n : bouton KEY0,
- position : interrupteurs SW,
- commande : sortie GPIO sur **PIN_V10**.

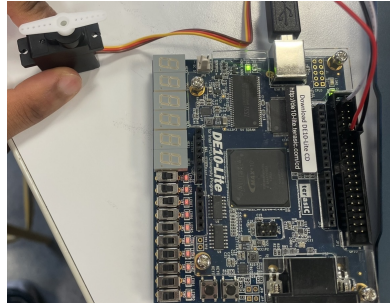
Mesures à l’oscilloscope L’observation du signal sur la broche PIN_V10 a permis de vérifier :

- 1 ms pour position = 0°,
- 1.5 ms pour position = 45°,
- 2 ms pour position = 90°.

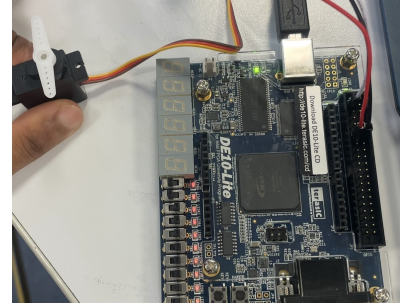
Test avec servomoteur réel Une fois connecté au signal `commande`, le servomoteur s'est correctement positionné aux angles demandés à l'aide des switches, validant l'implémentation.



(a) Montage - Position 0°



(b) Montage - Position 90°



(c) Montage - Position 180°

FIGURE 8 – Montage expérimental du servomoteur sur DE10-Lite.

4 Étape 2.3 : Extension de l'IP Servomoteur vers une Interface Avalon-MM

Dans cette partie, l'objectif est d'adapter l'IP Servomoteur précédemment développée afin qu'elle soit compatible avec le bus **Avalon-MM** et puisse être pilotée directement par le processeur Nios II au sein du système SoPC.

4.1 Spécifications de l'Interface Avalon

La nouvelle entité VHDL doit prendre en compte les signaux suivants :

Entrées :

- `chipselct` : sélection du périphérique Avalon,
- `write_n` : signal d'autorisation d'écriture (actif à 0),
- `reset_n` : remise à zéro active à l'état bas,
- `clk` : horloge système (50 MHz),
- `WriteData(31..0)` : bus d'écriture pour la commande du servomoteur.

Sortie :

- `commande` : signal PWM destiné au servomoteur.

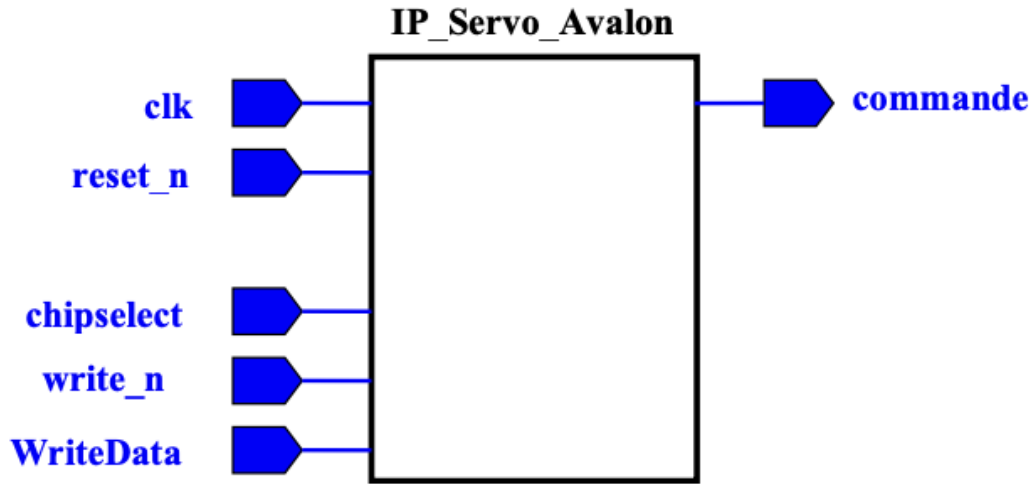


FIGURE 9 – Entité IP servo Avalon

4.2 2.3.1 Architecture et Développement de l'IP

Afin d'intégrer le contrôleur de servomoteur dans le système Nios II, l'IP a été étendue pour inclure une interface de registres conforme au standard Avalon-MM.

L'architecture finale repose sur trois blocs principaux :

1. Interface de registres Avalon

Un processus synchrone gère l'écriture du bus `WriteData`. Lorsque :

$$chipselect = 1 \quad \text{et} \quad write_n = 0$$

alors la valeur écrite par le processeur est stockée dans un registre interne :

$$reg_position \leftarrow WriteData(7..0)$$

Ce registre contient l'angle demandé ($0^\circ - 180^\circ$).

2. Calcul du rapport cyclique (mise à jour)

Un processus combinatoire convertit la position stockée (en degrés) en un temps d'impulsion exprimé en cycles d'horloge.

Avec une horloge à 50 MHz :

$$0,5 \text{ ms} = 25\,000 \text{ cycles} \quad \text{et} \quad 2,5 \text{ ms} = 125\,000 \text{ cycles}$$

La conversion linéaire utilisée devient :

$$Cycles = 25\,000 + Angle \times \frac{125\,000 - 25\,000}{180}$$

soit, en simplifiant :

$$Cycles = 25\,000 + Angle \times \frac{100\,000}{180}$$

Le facteur $\frac{100\,000}{180} \approx 555,56$ cycles par degré.

3. Générateur PWM (inchangé)

Un compteur libre génère une période fixe de 20 ms :

$$20\text{ ms} = 1\,000\,000\text{ cycles}$$

Le signal `commande` est défini par :

$$\begin{aligned} \text{commande} &= 1 && \text{si } \text{compteur} < \text{Cycles} \\ \text{commande} &= 0 && \text{sinon} \end{aligned}$$

Ainsi, la largeur d’impulsion varie automatiquement en fonction de la valeur écrite via Avalon.

4.3 2.3.2 Simulation et Validation (mise à jour)

Afin de vérifier le bon fonctionnement de l’interface Avalon-MM et la génération correcte du PWM, un banc de test dédié (`tb_IP_Servo_Avalon.vhd`) a été réalisé sous ModelSim.

Protocole de test (mise à jour)

Le testbench simule le comportement du processeur Nios II :

1. Mise à zéro du système (`reset_n = 0`).
2. Écriture de la valeur **0** : impulsion attendue de **0.50 ms**.
3. Écriture de la valeur **90** : impulsion attendue de **1.50 ms**.
4. Écriture de la valeur **180** : impulsion attendue de **2.50 ms**.

Résultats de simulation (mise à jour)

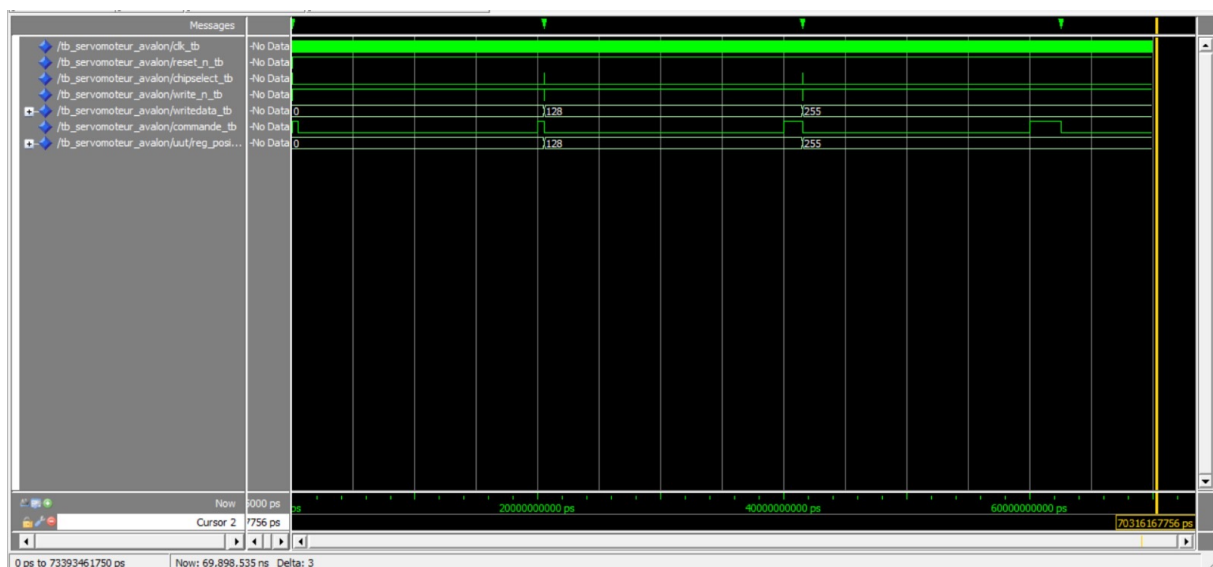


FIGURE 10 – Simulation ModelSim de l’IP Servomoteur avec interface Avalon plage d’impulsion 0,5–2,5 ms).

Analyse des résultats (mise à jour) Sur la chronologie de simulation, on vérifie :

- la séquence d'écriture Avalon (transition `write_n = 0`) et la mise à jour du registre `reg_position`,
- après écriture de 0 : largeur d'impulsion mesurée $\approx 0,50$ ms (25 000 cycles),
- après écriture de 128 : largeur d'impulsion mesurée $\approx 1,50$ ms (75 000 cycles),
- après écriture de 255 : largeur d'impulsion mesurée $\approx 2,50$ ms (125 000 cycles).

La formule mise à jour et l'implémentation arrondie assurent une correspondance correcte entre l'angle demandé et la largeur d'impulsion dans la nouvelle plage 0,5–2,5 ms, validant ainsi l'IP pour une intégration dans Platform Designer.

La capture montre clairement :

- les séquences d'écriture Avalon (transition `write_n = 0`),
- la mise à jour immédiate du registre interne `reg_position`,
- l'adaptation correcte de la largeur du PWM.

À chaque écriture :

- valeur 0 (0°) $\Rightarrow$ impulsion de 1 ms,
- valeur 128 (90°) $\Rightarrow$ impulsion d'environ 1.5 ms,
- valeur 255 (180°) $\Rightarrow$ impulsion de 2 ms.

L'IP réagit correctement aux signaux de contrôle du bus Avalon-MM, ce qui valide son intégration dans Platform Designer.

5 Étape 4 : Affichage des obstacles sur écran VGA

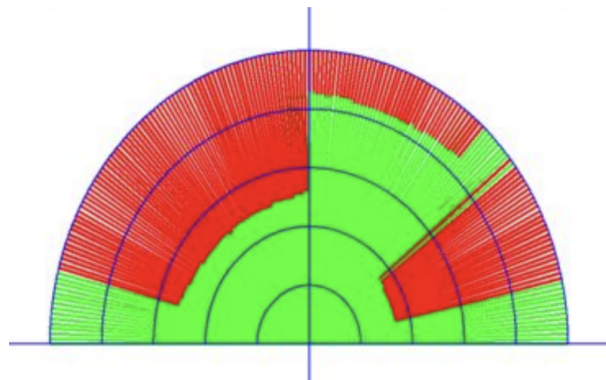


FIGURE 11 – Affichage attendu sur écran VGA

L'idée ici est de transformer les distances brutes reçues par le capteur ultrason en un affichage visuel sur l'écran VGA. Au lieu de simples chiffres, on crée une véritable interface radar en 2D : un balayage balaie la zone et affiche graphiquement la position des obstacles pour rendre la lecture plus intuitive.

5.1 Architecture du sous-système graphique

Pour la partie affichage, nous utilisons les composants de l'Intel FPGA University Program via Platform Designer. Le système repose sur deux blocs principaux :

Le VGA Controller : il gère la partie matérielle en générant les signaux de synchronisation (HSYNC/VSYNC) et les données RGB pour le DAC de la carte DE10-Lite.

Le Pixel Buffer : c'est notre mémoire vidéo. Chaque pixel de l'écran y est représenté par une couleur.

L'avantage de cette architecture est la séparation des tâches : le processeur Nios II se contente d'écrire les données graphiques dans le buffer, tandis que le contrôleur VGA s'occupe de rafraîchir l'écran en continu. Cela libère du temps de calcul et garantit un affichage fluide.

5.2 Transformation des coordonnées polaires vers le repère VGA

Le système nous renvoie des données sous forme polaire : une distance R pour chaque angle θ du servomoteur. Cependant, l'affichage VGA fonctionne sur un repère cartésien (pixels). Pour faire la correspondance, nous appliquons une transformation de coordonnées. En fixant le centre du radar au milieu du bas de l'écran (x_c, y_c), nous utilisons les formules de trigonométrie classiques pour convertir chaque détection en point (x, y) affichable :

$$x = x_c + R \cos(\theta)$$

$$y = y_c - R \sin(\theta)$$

Le signe négatif devant le terme en $\sin(\theta)$ est dû à l'orientation de l'axe vertical dans le repère VGA, où les coordonnées y augmentent vers le bas de l'écran.

Cette transformation permet de positionner correctement chaque point détecté dans l'espace d'affichage.

5.3 Résultats expérimentaux

L'affichage obtenu correspond à une vue radar en éventail. Les obstacles situés devant le capteur apparaissent sous forme de points rouges, tandis que le faisceau de balayage est matérialisé par une ligne verte animée.

La fluidité de l'affichage dépend principalement :

- de la vitesse de rotation du servomoteur,
- du temps d'attente minimal de **60 ms** requis entre deux mesures ultrasoniques afin d'éviter les échos parasites.

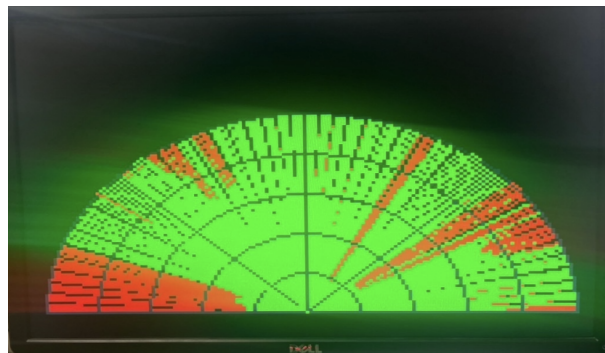


FIGURE 12 – Affichage radar VGA du système en fonctionnement.

Cette étape valide la chaîne complète d'acquisition, de traitement et de visualisation des données du radar.